```
import gpytorch


class BaseGPModel(gpytorch.models.ExactGP):
    def __init__(self, train_x, train_y, likelihood):
        ...    ❶

    def forward(self, x):
        ...    ❶
```

❶ To be discussed shortly.

Here, we implement a class called `BaseGPModel`, which has two specific methods: `__init__()` and `forward()`. The behavior of our GP model heavily depends on how we write these two methods, and no matter what kind of GP model we'd like to implement, our model class needs to have these methods.

Let's talk about the `__init__()` method first. Its job is to take in the training data set defined by the first and second arguments, `train_x` and `train_y`, as well as a likelihood function stored in the variable `likelihood`, and initialize the GP model, which is a `BaseGPModel` object. We implement the method as follows:

```
def __init__(self, train_x, train_y, likelihood):
    super().__init__(train_x, train_y, likelihood)
    self.mean_module = gpytorch.means.ZeroMean()
    self.covar_module = gpytorch.kernels.RBFKernel()
```

Here, we simply pass the three input arguments to the `__init__()` method of our super class, and the built-in implementation of `gpytorch.models.ExactGP` will take care of the heavy lifting for us. What remains is the definition of the mean and the covariance functions, which as we have said are the two main components of a GP.

In GPyTorch, there are a wide range of choices for both the mean and the covariance function, which we will explore in Chapter 3. For now, we will use the most common options for a GP:

- `gpytorch.means.ZeroMean()` for the mean function, which outputs zero mean predictions in prior mode, and
- `gpytorch.kernels.RBFKernel()` for the covariance function, which implements the *Radial Basis Function* (RBF) kernel, which is one of the most commonly used covariance function for GPs and implements the idea that data points that are close to each other are correlated to each other.

We store these objects in the `mean_module` and `covar_module` class attributes, respectively. That's all we need to do for the `__init__()` method. Now, let's turn our attention to the `forward()` method.

The `forward()` method is very important, as it defines how the model should process its input. If you have worked with neural networks in PyTorch, the `forward()` method of a network class

©Manning Publications Co. To comment go to liveBook
https://livebook.manning.com/#!/book/bayesian-optimization-in-action/discussion

Licensed to ashwin sampath <ashwin.manning@gmail.com>